

T1A1 — Five-Point Finite-Difference Formulas

2026-09-21 · @Someone

Physics Modelling and Simulations: five-point central-difference approximations for f' and f'' , derived via Taylor expansion and implemented in Python.

a) Five-point formula for the first derivative

Start from the Taylor expansions of f about x_i :

$$f_{i+1} - f_{i-1} = 2hf_i' + \frac{h^3}{3}f_i^{(3)} + O(h^5) \quad \text{tag}\{1\}$$

$$f_{i+2} - f_{i-2} = 4hf_i' + \frac{8h^3}{3}f_i^{(3)} + O(h^5) \quad \text{tag}\{2\}$$

Multiplying (1) by 8 makes the third-derivative coefficients equal, so subtracting (2) cancels the $f_i^{(3)}$ term:

$$8f_{i+1} - 8f_{i-1} - (f_{i+2} - f_{i-2}) = 12hf_i' + O(h^5) \quad \text{tag}\{3\}$$

Dividing by $12h$ gives the five-point formula:

$$f_i' = \frac{1}{12h} (f_{i-2} - 8f_{i-1} + 8f_{i+1} - f_{i+2}) + O(h^4) \quad \text{tag}\{4\}$$

Since the error before division was $O(h^5)$, dividing by a factor proportional to h brings it to $O(h^4)$ — one order better than the standard three-point central difference.

b) Five-point formula for the second derivative

This time keep the even-order terms by adding, rather than subtracting, the neighbouring expansions:

$$f_{i+1} + f_{i-1} = 2f_i + h^2f_i'' + \frac{h^4}{12}f_i^{(4)} + O(h^6) \quad \text{tag}\{5\}$$

$$f_{i+2} + f_{i-2} = 2f_i + 4h^2f_i'' + \frac{4h^4}{3}f_i^{(4)} + O(h^6) \quad \text{tag}\{6\}$$

Multiplying (5) by 16 matches its $f_i^{(4)}$ coefficient to that of (6), so subtracting (6) cancels the fourth-derivative term:

$$16\left(f_{i+1}+f_{i-1}\right) - \left(f_{i+2}+f_{i-2}\right) = 30f_i + 12h^2f_i'' + O(h^6) \quad \text{\tag{7}}$$

Rearranging and dividing by $12h^2$:

$$f_i'' = \frac{1}{12h^2}\left(-f_{i-2}+16f_{i-1}-30f_i+16f_{i+1}-f_{i+2}\right) + O(h^4) \quad \text{\tag{8}}$$

As before, the $O(h^6)$ error is divided by h^2 , giving $O(h^4)$ overall — the same order of accuracy as the first-derivative formula.

Python implementations

Both formulas need the two neighbours on each side, so the endpoints (first and last two points) have no valid five-point stencil and are left as NaN.

```
import numpy as np

def five_point_first_derivative(f, h):
    """Eq. (4): f_i' from f_{i-2..i+2}, O(h^4) accurate."""
    fp = np.full_like(f, np.nan, dtype=float)
    fp[2:-2] = (f[:-4] - 8*f[1:-3] + 8*f[3:-1] - f[4:]) / (12*h)
    return fp

def five_point_second_derivative(f, h):
    """Eq. (8): f_i'' from f_{i-2..i+2}, O(h^4) accurate."""
    fpp = np.full_like(f, np.nan, dtype=float)
    fpp[2:-2] = (-f[:-4] + 16*f[1:-3] - 30*f[2:-2] + 16*f[3:-1] - f[4:]) /
    (12*h**2)
    return fpp
```

`f` is a 1D array of function values on a uniform grid of spacing `h`; both functions are fully vectorized (no Python-level loop over grid points).